EventFlow — Participant & Class Reference

(Renamed: `Subject` → `EventSubject`, `Observer` → `EventObserver`, throughout.)

Part 1 — GoF Participants

Composite pattern

Participant	Class	Inherits from	Notes
Component	`EventComponent`	(abstract base, no parent)	Defines the interface every part of the tree must support
Leaf	`EventUnit` (abstract) → `Stage`, `Gate`, `Vendor`, `MedicalTeam`, `ShuttleStop`, `InfoDesk` (concrete)	`EventUnit` inherits from `EventComponent`	Leaves have no children; each concrete leaf is a "primitive object" of the tree
Composite	`EventGroup` and its subclass `RelayGroup`	`EventGroup` inherits from `EventComponent`. `RelayGroup` inherits from `EventGroup` (not directly from `EventComponent`).	Holds children, implements operations recursively
Client	`main.cpp`	—	Builds the tree (calls `add()`/`remove()`) and invokes operations through the `EventComponent` interface without knowing if it's a Leaf or a Composite. `EventControl` is *not* this Client — it never builds or

Participant	Class	Inherits from	Notes
			touches the ownership tree; see the note below.

> **Note:** `EventControl` plays no role at all in the Composite pattern — it's not a Component, Leaf, Composite, or Client. It only shows up in the *Observer* table below, as a ConcreteSubject. A class doesn't have to appear in every pattern; `EventControl` exists purely to issue notices.

Observer pattern

Participant	Class	Inherits from	Notes
EventSubject	`EventSubject` (abstract)	(abstract base, no parent)	Declares attach/detach/notify
ConcreteSubject	`EventGroup`, `RelayGroup` (gets it transitively), `EventControl`	`EventGroup` inherits from `EventSubject` (direct). `RelayGroup` inherits from `EventGroup` (so it gets `EventSubject` *through* `EventGroup`, not directly). `EventControl` inherits from `EventSubject` (direct, completely separate class hierarchy — has nothing to do with `EventGroup`).	Actually stores the observer list and fires `notify()`
EventObserver	`EventObserver` (abstract)	(abstract base, no parent)	Declares `update()`
ConcreteObserver	`RelayGroup`, `Stage`, `Gate`, `Vendor`, `MedicalTeam`,	`RelayGroup` inherits from `EventGroup` AND `EventObserver`	Implements `update()` with its own reaction

Participant	Class	Inherits from	Notes
	`ShuttleStop`, `InfoDesk`	(two separate base classes). `Stage`, `Gate`, `Vendor`, `MedicalTeam`, `ShuttleStop`, `InfoDesk` each inherit from `EventUnit`. `EventUnit` inherits from `EventComponent` AND `EventObserver` (two separate base classes).	

Classes that wear two hats (and why that's intentional, not a mistake)

Class	Roles	Why it's not a misuse
`RelayGroup`	Composite (via `EventGroup`) **and** EventObserver	It's contained in the tree (Composite collaboration) **and separately** registered to hear notices from whatever is above it (Observer collaboration). Two different relationships, two different reasons.
`EventGroup`	Composite **and** EventSubject	It owns/organises children (Composite) **and separately** can broadcast a notice to whoever registered with it (Observer's Subject side).
`EventUnit` (all leaves)	Leaf **and** EventObserver	It's a primitive tree node (Composite) **and separately** reacts to notices it's registered for (Observer).

Everything else (`EventComponent`, `EventSubject`, `EventObserver`, `EventControl`, `Notice`) plays exactly **one** role.

Part 2 — UML class boxes (copy-paste ready)

Notation used below (matches standard 3-compartment UML class notation):

- Class name: **bold, centred**; *italic* if the class is abstract.
- Attributes: `visibility name: type = defaultValue` (default value only where one makes sense).
- Operations: `visibility name(parameter: type): returnType`.
- **Pure virtual** operations are set `= 0` and written in *italics*.
- **All virtual** operations (pure or overriding) are written in *italics* — non-virtual, ordinary operations are plain.
- `+` public, `-` private, `#` protected.

EventComponent — abstract

Attributes

name: string

capacity: int

isOpen: bool = false

Operations

+ EventComponent(name: string, capacity: int)

+ open(): void = 0 (italic)

+ close(): void = 0 (italic)

+ reportStatus(): void = 0 (italic)

+ getCapacity(): int = 0 (italic)

+ getName(): string

+ ~EventComponent(): void (italic — virtual)

EventSubject — abstract

Operations

+ attach(observer: EventObserver*): void = 0 (italic)

+ detach(observer: EventObserver*): void = 0 (italic)

+ notify(notice: Notice&): void = 0 (italic)

+ ~EventSubject(): void (italic — virtual)

EventObserver — abstract

Operations

+ update(notice: Notice&): void = 0 (italic)

+ ~EventObserver(): void (italic — virtual)

NoticeType — enumeration (not a class box)

OPEN, CLOSE, SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT, PAUSE, RESUME, EVACUATE

Notice — concrete

Attributes

+ type: NoticeType

+ message: string

+ severity: int = 0

Operations

+ Notice(type: NoticeType, message: string, severity: int)

EventUnit — abstract *(inherits EventComponent, EventObserver)*

Operations

+ EventUnit(name: string, capacity: int)

+ open(): void (italic — virtual, gives shared default)

+ close(): void (italic — virtual)

+ reportStatus(): void (italic — virtual)

+ getCapacity(): int (italic — virtual)

+ update(notice: Notice&): void = 0 (italic — still pure here)

+ ~EventUnit(): void (italic — virtual)

Stage — concrete *(inherits EventUnit)*

Attributes

- isPerformancePaused: bool = false

Operations

+ Stage(name: string, capacity: int)

+ open(): void (italic — virtual)

+ close(): void (italic — virtual)

+ update(notice: Notice&): void (italic — virtual)

+ ~Stage(): void (italic — virtual)

Gate — concrete *(inherits EventUnit)*

Attributes

- isAdmitting: bool = true

Operations

+ Gate(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual)

+ ~Gate(): void (italic — virtual)

Vendor — concrete *(inherits EventUnit)*

Attributes

- isServing: bool = true

Operations

+ Vendor(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual)

+ ~Vendor(): void (italic — virtual)

MedicalTeam — concrete *(inherits EventUnit)*

Attributes

- readinessLevel: int = 0

Operations

+ MedicalTeam(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual)

+ ~MedicalTeam(): void (italic — virtual)

ShuttleStop — concrete *(inherits EventUnit)*

Attributes

- currentRoute: string

Operations

+ ShuttleStop(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual)

+ ~ShuttleStop(): void (italic — virtual)

InfoDesk — concrete *(inherits EventUnit)*

Attributes

- currentSignage: string

Operations

+ InfoDesk(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual)

+ ~InfoDesk(): void (italic — virtual)

EventGroup — concrete *(inherits EventComponent, EventSubject)*

Attributes

- children: vector<EventComponent*>

- observers: vector<EventObserver*>

Operations

+ EventGroup(name: string, capacity: int)

+ add(component: EventComponent*): void (plain — new operation, not overriding anything)

+ remove(component: EventComponent*): void (plain)

+ getChild(index: int): EventComponent* (plain)

+ open(): void (italic — virtual override)

+ close(): void (italic — virtual override)

+ reportStatus(): void (italic — virtual override)

+ getCapacity(): int (italic — virtual override)

+ attach(observer: EventObserver*): void (italic — virtual override)

+ detach(observer: EventObserver*): void (italic — virtual override)

+ notify(notice: Notice&): void (italic — virtual override)

+ ~EventGroup(): void (italic — virtual)

RelayGroup — concrete *(inherits EventGroup, EventObserver)*

Operations

+ RelayGroup(name: string, capacity: int)

+ update(notice: Notice&): void (italic — virtual override)

+ ~RelayGroup(): void (italic — virtual)

EventControl — concrete *(inherits EventSubject)*

Attributes

- observers: vector<EventObserver*>

Operations

+ EventControl()

+ attach(observer: EventObserver*): void (italic — virtual override)

+ detach(observer: EventObserver*): void (italic — virtual override)

+ notify(notice: Notice&): void (italic — virtual override)

+ issueNotice(type: NoticeType, message: string, severity: int): void (plain — new operation)

+ ~EventControl(): void (italic — virtual)

Quick mental model

- **Two trees, not one:** the ownership tree (`children`, black-diamond, `EventGroup`/`RelayGroup` → `EventComponent`) and the notification graph

(`observers`, plain arrow, any `EventSubject` → any `EventObserver`) are separate data structures that happen to overlap in places.

- **Only `RelayGroup` and `EventControl` ever call `notify()`.** Plain `EventGroup` *can* (it inherits the method), but in this design it's only actually used by the nodes that sit in a real cascade chain.
- **Only `RelayGroup` and the six leaves ever have `update()` called on them** — because they're the only classes that implement `EventObserver`.
- **Why so many things are italic:** in C++, once a function is declared `virtual` in a base class, every override down the hierarchy is still virtual (that's what makes polymorphism work) — even a concrete override in a leaf class. So italics track "is this dispatched polymorphically", not "is this still unimplemented."